Capstone Project 2

Event-Driven Order Processing Platform (SQS + SNS + Lambda + DynamoDB + API Gateway)

Industry Scenario

An **E-commerce platform** handling millions of orders per day.

System must:

- Be asynchronous
- Highly scalable
- Fault tolerant
- Event-driven

Problem Statement

Design an event-driven order processing system:

- API Gateway receives orders
- Lambda validates order
- SNS broadcasts event
- SQS queues for fulfillment & billing
- DynamoDB stores order state
- KMS encryption enabled

Use Cases

1. Place Order
2. Payment processing
3. Inventory deduction
4. Notification emails

Architecture Components

- API Gateway
- Lambda
- DynamoDB

- SNS
- SQS
- KMS
- CloudWatch

Terraform Code

1 DynamoDB

```
resource "aws_dynamodb_table" "orders" {
  name           = "Orders"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "order_id"

  attribute {
    name = "order_id"
    type = "S"
  }
}
```

---

2 SNS Topic

```
resource "aws_sns_topic" "order_topic" {
  name = "order-events"
}
```

---

3 SQS Queue

```
resource "aws_sqs_queue" "fulfillment_queue" {
  name = "fulfillment-queue"
}
```

---

4 Lambda Function

```
resource "aws_lambda_function" "order_lambda" {
  function_name = "order-handler"
  handler       = "index.handler"
  runtime       = "python3.9"
  role          = aws_iam_role.lambda_role.arn
  filename      = "lambda.zip"
}
```

---

5 API Gateway

```
resource "aws_api_gateway_rest_api" "orders_api" {
  name = "orders-api"
}
```